# CADENCE: Case-Grounded, Deliberation-Driven, and Solver-Verified Architectural Decision-Making with Large Language Models

Quang-Vinh Dang

***Abstract*—Large language models (LLMs) are increasingly proposed as assistants for architectural decision-making in service-oriented systems, yet most existing approaches either generate an architecture decision record (ADR) from a single unconstrained LLM call, or retrieve similar precedents without reconciling the competing quality-attribute concerns a real decision must balance, or debate among multiple agents without any formal check that the resulting decision is actually realizable within a bounded budget of architectural tactics. We present CADENCE, a four-stage algorithm that combines (1) retrieval-augmented case-based reasoning over a corpus of real, mined ADRs, (2) knowledge-graph-grounded multi-agent deliberation in which each agent advocates for one ISO/IEC 25010 quality attribute, (3) constraint-solver-verified feasibility, encoded as a two-phase weighted MaxSAT/SMT problem in Z3, with an LLM repair loop that targets the solver's unsatisfiable core, and (4) a separate self-critique finalization pass that blends a deterministic structural utility score with a qualitative LLM judgment. Unlike prior multi-agent architectural decision systems, whose agents occupy sequential functional roles and whose "evaluation" step is itself another LLM opinion, CADENCE's agents represent adversarial quality-attribute concerns and its feasibility check is a formal solver, not a second language model. We implement the full pipeline against a real corpus of 6,173 mined ADRs and evaluate it against three baselines — zero-shot generation, retrieval-only generation, and multi-agent deliberation without solver verification — on BERTScore, BLEU, ROUGE-1, and METEOR, together with two metrics no solver-free baseline can report: constraint-satisfaction rate and repair-loop convergence. We find that surface n-gram metrics penalize CADENCE's deliberately terse, solver-parseable output relative to baselines prompted for open-ended prose, while BERTScore — a semantic-similarity metric insensitive to this length mismatch — remains comparable across all systems; we argue this asymmetry, not a quality gap, and discuss its implications for evaluating structured multi-stage LLM pipelines in general.**

***Index Terms*—Large language models, architectural decision-making, architecture decision records, multi-agent systems, constraint solving, retrieval-augmented generation, self-critique, service-oriented architecture.**

## I. Introduction

Service-oriented systems accumulate architectural decisions continuously: how to scale a read-heavy service, whether to introduce a message queue, where to draw a service boundary, how to trade encryption overhead against latency. Each such decision commits a team to a set of consequences across multiple, frequently conflicting quality attributes — performance against maintainability, scalability against cost, security against usability — and each is, in practice, made under time pressure, with incomplete visibility into precedent, and without a systematic check that the chosen combination of architectural tactics is even mutually consistent within whatever operational budget the team can actually staff and afford. Architecture decision records (ADRs) exist precisely to make this reasoning explicit and reviewable, yet the discipline of writing one well — surveying precedent, weighing competing quality attributes on their merits, and verifying that the resulting commitments do not silently exceed what the team can deliver — is exactly the kind of effortful, multi-perspective reasoning that teams under delivery pressure are most likely to shortcut.

Large language models are a natural candidate to assist here, and a growing body of work has begun to explore LLM-assisted architectural decision-making and ADR generation. The most direct approaches prompt a single LLM, zero-shot or with light context, to draft a decision from a stated set of requirements [1]. A second line of work grounds generation in retrieval over real, mined ADR corpora, so that the model's output is anchored in precedent rather than invented from parametric knowledge alone [2]. A third line introduces multiple LLM agents into the process, most notably a four-agent pipeline of functional roles — an Analyst, a Modeler, a Designer, and an Evaluator — each handling a different phase of the decision [4]. Each of these three lines of work solves part of the problem this paper is concerned with, but none solves all of it at once, and — more importantly — none of them can answer a question that matters for any decision with real operational consequences: *is the decision the model just wrote actually feasible, given a bounded and explicit budget of architectural commitments the team can realistically execute?* A single LLM call has no mechanism to check this. Retrieval grounds the decision in precedent but does not reconcile competing quality-attribute demands against each other, let alone verify them formally. And a multi-agent pipeline whose final "Evaluator" step is itself another LLM call is still, ultimately, one model's opinion about another model's output — persuasive, perhaps, but not a proof of anything.

Q.-V. Dang is with British University Vietnam, Hung Yen, Vietnam (e-mail: vinh.dq4@buv.edu.vn).

Manuscript submitted to the IEEE Transactions on Services Computing Special Issue on "Large Language Models in Service-Oriented Ecosystems Design: Advances and Applications," 2026.

This paper presents CADENCE, a four-stage algorithm for LLM-assisted architectural decision-making that is designed around this gap directly. CADENCE (1) retrieves precedent decisions from a real corpus of mined, human-authored ADRs via dense vector search, so that deliberation starts from what teams have actually decided before, not from a blank context window; (2) stages a bounded, structured multi-agent deliberation in which each agent is anchored to one ISO/IEC 25010 quality attribute — performance, security, maintainability, scalability, and cost/operability — and argues for the candidate decision that best serves its attribute, grounded in a knowledge graph of architectural tactics and their known cross-attribute trade-offs, so that the competing concerns a real decision must balance are represented explicitly and adversarially rather than folded into one model's undifferentiated judgment; (3) encodes the deliberated candidate's implied tactic commitments as a two-phase weighted MaxSAT/SMT problem and checks it with the Z3 theorem prover, and, if the encoding is infeasible under a caller-specified tactic budget, hands the solver's unsatisfiable core back to the agents as a targeted repair prompt, iterating to a fixed cap and degrading gracefully with an explicit caveat if feasibility is never reached; and (4) finalizes the solver-verified decision through a separate self-critique pass that combines a deterministic structural utility score — derived directly from which quality attributes the selected tactics actually cover, and what trade-offs they incur — with a qualitative LLM judgment of the decision's remaining weaknesses, so that the system's own claimed confidence is never purely self-reported by the same reasoning process that produced the decision. Each stage exists to close a specific failure mode the others leave open: retrieval alone cannot reconcile competing quality attributes; deliberation alone cannot verify feasibility; and an unverified, unaudited decision cannot be trusted as final without an independent finalization pass.

We implement the complete CADENCE pipeline end-to-end against a real, publicly available corpus of 6,173 mined ADRs drawn from 883 open-source repositories [6], and evaluate it against three baselines that isolate each of CADENCE's contributions in turn: zero-shot generation (no retrieval, no deliberation, no verification), retrieval-only generation (retrieval without deliberation or verification), and multi-agent deliberation without solver verification (the ablation that most directly probes whether the constraint-solver stage adds anything beyond what unverified multi-agent debate already provides). We report the standard generation-quality metrics used in prior ADR-generation work — BERTScore, BLEU, ROUGE-1, and METEOR — against held-out, human-authored ground truth, alongside two metrics that only a solver-verified system can report at all: constraint-satisfaction rate and repair-loop convergence. Our results surface a methodologically important asymmetry: surface n-gram overlap metrics (ROUGE-1, METEOR) penalize CADENCE's deliberately terse, structured output — terse by design, so that Stages 3 and 4 can parse it reliably — relative to baselines prompted for open-ended prose closer in length to the reference ADRs, while BERTScore, a semantic-similarity metric far less sensitive to this length mismatch, remains comparable across all four systems. We report this honestly rather than letting a metric artifact of our own prompt design misrepresent the comparison, and we discuss what it implies for evaluating structured, multi-stage LLM pipelines whose intermediate representations are deliberately not free-form prose.

This work makes three contributions. First, we present CADENCE, to our knowledge the first architectural decision-making algorithm to combine real-corpus retrieval, adversarial quality-attribute-grounded multi-agent deliberation, formal constraint-solver verification with a solver-in-the-loop repair mechanism, and a separately-scored self-critique finalization stage. Second, we give a concrete, empirically-validated construction for turning free-text, LLM-authored architectural deliberation into a solvable constraint-satisfaction instance — including the specific, verified fix (lexicographic rather than weighted-sum multi-objective optimization) required to make quality-attribute coverage strictly dominate trade-off avoidance regardless of caller-supplied weight magnitudes, a subtlety that a naive encoding gets wrong. Third, we report a full empirical evaluation against real baselines on a real corpus, and surface and explain a metric asymmetry (BERTScore versus surface n-gram overlap) that we believe generalizes to other structured, multi-stage LLM systems whose output is deliberately not free-form prose.

The remainder of this paper is organized as follows. Section II positions CADENCE against the closest prior systems for LLM-assisted architectural decision-making and against the broader techniques it draws on. Section III presents the CADENCE algorithm in full, stage by stage. Section IV describes our experimental setup, baselines, metrics, and results. Section V discusses limitations, threats to validity, and the metric-asymmetry finding. Section VI concludes.

## II. Related Work

### A. LLM-Assisted Architectural Decision-Making

Dhar et al. [1] demonstrate that a single LLM, prompted with a stated architectural problem, can produce a plausible ADR, but their approach neither retrieves precedent nor verifies the result against any formal notion of feasibility: the model's output is accepted as-is. Follow-on work from the same group frames ADR authoring as an explicit drafting process rather than one-shot generation [7], and a separate recent study evaluates LLMs specifically as *detectors* of violations against already-recorded architectural decisions [8] — a complementary, downstream concern (checking compliance with a decision already made) rather than assisting the decision itself. A complementary line of work treats the growing body of real, mined ADRs as a resource in its own right rather than a target for generation: large-scale mining studies of ADR adoption in open-source projects [6] establish both the raw corpus this paper's retrieval stage draws on and the empirical observation — corroborated in our own corpus statistics — that ADR adoption per repository is sparse, which any retrieval-based system must be designed to tolerate rather than assume away. Building on such corpora, retrieval-grounded generation work [2], [3] shows that conditioning an LLM's ADR generation on retrieved precedent measurably improves output quality over unconditioned generation, and explicitly frames the task this paper's held-out evaluation also uses: given only a decision's title, generate the body. This precedent motivates our own held-out query construction (Section IV) and supplies both the retrieval-only baseline our evaluation compares against directly and the corpus itself. Neither line of work, however, reconciles multiple quality-attribute concerns against each other or checks feasibility formally — both are gaps CADENCE is designed to close.

The closest prior system to CADENCE is MAAD [4], a four-agent architectural decision-making pipeline in which an Analyst, a Modeler, a Designer, and an Evaluator agent each handle one sequential phase of the decision process; a related paper from an overlapping author group targets the adjacent problem of generating the *rationale* behind an already-made decision rather than the decision itself [5]. MAAD and CADENCE share the premise that a single LLM call is insufficient and that multiple agents should participate in producing an architectural decision, but the two systems differ in three concrete, load-bearing ways. First, MAAD's agents occupy sequential *functional* roles — each agent does a different *kind* of work at a different point in a pipeline — whereas CADENCE's agents occupy simultaneous, *adversarial* roles: every agent does the same kind of work (propose and critique a candidate decision) but from the standpoint of a different, often conflicting quality attribute, so that the tension between (for instance) a performance-motivated tactic and its maintainability cost is represented as an argument between agents rather than resolved silently inside one agent's internal reasoning. Second, MAAD's Evaluator is itself an LLM judge: it assesses the Designer's output using the same kind of model that produced it, so MAAD's final confidence signal is, ultimately, one language model's opinion of another's work. CADENCE's Stage 3 feasibility check is a formal constraint solver over an explicit tactic budget and quality-attribute coverage requirement; a decision either satisfies the encoded constraints or it does not, independent of any model's subjective assessment, and when it does not, the solver's unsatisfiable core gives the repair loop a structured, specific target rather than generic revision instructions. Third, CADENCE's retrieval stage is grounded in a large, real, citable corpus of mined ADRs from open-source practice, giving deliberation a case-based starting point that MAAD's requirements-only formulation does not provide. We discuss MAAD further, and compare against a solver-free multi-agent ablation modeled directly on this difference, in Section IV.

### B. LLMs for Service-Oriented and Microservice Architecture

This paper's special-issue theme — LLMs in service-oriented ecosystems design — has its own emerging, directly relevant literature that CADENCE's problem framing builds on. A recent review positions LLMs across the service-computing lifecycle broadly, from discovery and composition to design assistance, and identifies architectural decision support as an open direction rather than a solved problem [11]. More specifically to design synthesis, recent work uses contrastive learning alongside LLMs to recommend monolith-to-microservice decompositions [12], and a complementary evaluation synthesizes candidate microservice architectures directly from textual requirements [13]. These systems target a structurally different problem from CADENCE's — decomposing or synthesizing a service topology, rather than deciding and justifying one architectural commitment against competing quality-attribute concerns — but they confirm that LLM-assisted service-oriented design is an active, contested area in which no prior system combines case-grounded retrieval, adversarial multi-agent deliberation, and formal feasibility verification, the specific combination CADENCE contributes.

### C. LLM–Solver Integration

Combining LLMs with formal solvers to check or verify what a model proposes is an active area outside the architecture domain: representative approaches translate natural-language problems into a formal logical representation and dispatch it to a symbolic solver so that the solver, not the LLM, is responsible for the deductive step [14], and closely related work verifies LLM-authored natural-language proofs by first compiling them into a checkable formal representation [15]. More recent work bridges LLMs and symbolic solvers directly through the Model Context Protocol, giving a model a structured tool interface to a solver rather than a one-shot translation step [16], and, closest in spirit to CADENCE's Stage 3, one planning system explicitly drives an LLM repair loop from a Z3 solver's unsatisfiable core when a proposed plan is infeasible [17] — the same solver-in-the-loop repair pattern this paper applies to architectural tactic feasibility rather than task planning. These techniques target logical inference, formal proof checking, and classical planning; CADENCE applies the same underlying pattern — let the LLM produce a candidate, let a solver check it, and feed the solver's failure signal back to the LLM for repair — to a different, softer verification target: whether a candidate architectural decision's implied tactic commitments are jointly satisfiable within an explicit resource budget and quality-attribute coverage requirement, encoded as weighted MaxSAT/SMT rather than propositional or first-order logic or a planning domain. To our knowledge, no prior work applies solver-in-the-loop verification specifically to architectural trade-off feasibility; we cite this literature as prior art for the general LLM-plus-solver *technique*, not as prior work on the specific *application*.

### D. Multi-Agent Deliberation and Self-Critique with LLMs

Multi-agent debate among LLM instances has been shown to improve factual accuracy and reasoning quality over single-model generation by exposing a claim to independent critique before it is accepted [18], an effect corroborated by independent work showing debate encourages divergent thinking that a single model's self-reflection alone does not reliably produce [19], and extended to using multi-agent debate itself as an evaluation mechanism for other models' outputs [20]. CADENCE's Stage 2 deliberation applies this general finding to a domain-specific structure: rather than generic, undifferentiated debate, each agent's role and critique perspective is fixed to one ISO/IEC 25010 quality attribute and grounded in a shared knowledge graph of architectural tactics and their documented trade-offs, so the debate is anchored to concrete, checkable claims rather than open-ended argument. Separately, self-refinement work shows that a model can improve its own output by critiquing and revising it in a subsequent pass, without external supervision [21], and verbal-reinforcement approaches extend this across multiple attempts using episodic memory of past failures [22]. This literature is not uniformly optimistic, however: a widely-cited negative result finds that LLMs prompted to self-correct their own reasoning *without external feedback* frequently fail to improve, and can even degrade an already-correct answer [23] — a finding we take seriously and address directly in Stage 4's design (Section III) and in our threats-to-validity discussion (Section V). CADENCE's Stage 4 draws on the self-refinement idea but departs from both strands in one respect we consider important for high-stakes decisions: the self-critique pass is not a further revision of the same reasoning chain in isolation, but an independently-scored finalization that combines a deterministic structural component — computed directly from Stage 3's solver output, not re-derived by an LLM — with the qualitative LLM judgment, so the system's final confidence is never solely the same model marking its own homework without external (solver-verified) grounding.

### E. Case-Based Reasoning and Retrieval-Augmented Generation

Case-based reasoning — solving new problems by adapting solutions to similar past problems — is a natural fit for architectural decision-making, since real decisions are rarely made from first principles but from precedent, convention, and prior experience; recent work formalizes how case-based reasoning's classical case-retrieve-reuse-revise cycle maps onto LLM agents specifically [27], and a concrete instantiation combines case-based retrieval with generation for a different high-stakes domain (legal question answering), demonstrating the pattern's applicability beyond architecture [26]. CADENCE's Stage 1 operationalizes this classical idea with dense vector retrieval over a real ADR corpus rather than a hand-curated case base, following the same retrieval-augmented generation methodology increasingly used to ground LLM generation in verifiable external evidence more broadly [24], including within software engineering specifically, where a recent survey catalogs repository-level retrieval strategies for code generation that parallel this paper's corpus-level retrieval for architectural precedent [25]. The architectural tactics that structure CADENCE's knowledge graph and constraint encoding — caching, connection pooling, horizontal scaling, circuit breakers, and the like, together with their well-known cross-quality-attribute trade-offs — are drawn from the established software architecture tactics catalog [10], which this paper's Stage 2/3 machinery operationalizes into a form an LLM-driven deliberation and a formal solver can both act on.

## III. The CADENCE Algorithm

### A. Problem Formulation and Overview

CADENCE takes as input a natural-language decision context $c$ (the requirements and situational description a real ADR's "Context" section would state) and a set of required quality attributes $Q = \{q_1, \ldots, q_5\} \subseteq \{\mathit{performance}, \mathit{security}, \mathit{maintainability}, \mathit{scalability}, \mathit{cost\_operability}\}$, the ISO/IEC 25010-derived taxonomy fixed for this work. It produces a finalized, provenance-complete architectural decision record: a decision statement, a rationale, a per-quality-attribute utility score, a feasibility verdict with the selected architectural tactics and any residual gaps, and a full transcript of the precedents consulted, the agent positions debated, and the repair iterations taken to reach the final candidate. The four stages — retrieval, deliberation, solver verification with repair, and self-critique finalization — execute in sequence, each consuming the previous stage's output and each addressing a distinct failure mode the others do not cover.

### B. Stage 1: Retrieval-Augmented Case-Based Reasoning

Stage 1 embeds the decision context $c$ with a sentence-embedding model and retrieves the $k$ nearest precedent ADRs, by cosine similarity, from a dense vector index built over the full corpus of 6,173 real, mined ADRs (Section IV). Each corpus record is parsed leniently from its source Markdown file — real-world ADR filenames and internal structure are inconsistent across repositories, so the parser extracts only a title and the full raw body text rather than assuming any fixed section schema. Retrieval returns the top-$k$ precedent titles and bodies, which are passed into Stage 2 as grounding context for every deliberation agent; when Stage 1 is used inside a held-out evaluation, the query record's own ground-truth ADR is explicitly excluded from its own retrieval results to prevent leakage.

### C. Stage 2: Knowledge-Graph-Grounded Multi-Agent Deliberation

Stage 2 instantiates one `QualityAttributeAgent` per required quality attribute in $Q$. Each agent is grounded in a knowledge graph $G$ built from a hand-curated catalog of 25 architectural tactics spanning the five quality attributes (5 tactics per attribute), where each tactic node has a directed *supports* edge to the quality attribute it primarily serves and directed *trade-off* edges, annotated with a natural-language explanation, to every other quality attribute it is known to burden — for instance, *caching* supports *performance* but trades off against *maintainability* via cache-invalidation complexity, and *authentication* supports *security* but trades off against both *performance* (per-request verification latency) and *cost_operability* (identity-infrastructure upkeep). Given the decision context and Stage 1's retrieved precedents, each agent proposes a candidate decision serving its own attribute and may critique the candidates proposed by other agents, citing specific tactic trade-offs from $G$. Deliberation proceeds for a bounded number of rounds (`max_rounds`, an algorithm parameter) after which an orchestrator synthesizes the debate into a single converged candidate decision and rationale.

Synthesis is the stage's most format-fragile step: it asks the underlying LLM to emit a structured `CANDIDATE`/`RATIONALE` response that Stage 3 can parse deterministically. In practice, real small-model output deviates from any single expected format in ways a fixed parser cannot fully anticipate in advance (Section III-F); the orchestrator therefore retries the same synthesis prompt up to three times on a parse failure before surfacing an error, since generation is stochastic and a differently-sampled retry frequently self-corrects a formatting slip that a hand-written regex extension would not have anticipated.

### D. Stage 3: Constraint-Solver-Verified Feasibility with Repair Loop

Stage 2's converged candidate is free text: a human-readable decision and rationale, not a structured commitment a solver can reason about directly. Stage 3 first extracts which of the 25 catalog tactics the candidate text actually mentions, via fuzzy word-stem overlap against each tactic's name and description, giving a set of Boolean decision variables $\{t_1, \ldots, t_m\}$, one per mentioned tactic. It then checks feasibility in two phases against the Z3 SMT solver.

*Phase 1 (strict feasibility).* A budget constraint $\sum_i t_i \leq B$ (a caller-specified tactic budget $B$) and, for every required quality attribute $q \in Q$, a coverage constraint $\bigvee_{i:\mathrm{supports}(t_i)=q} t_i$ (at least one selected tactic must support $q$) are asserted with `assert_and_track`, so that if the instance is unsatisfiable, Z3 returns an *unsatisfiable core*: the specific subset of constraints jointly responsible for infeasibility. This core is a genuinely useful, but not necessarily complete, diagnostic — Z3's core-extraction semantics can name only one of several symmetric zero-coverage attributes as blocking, omitting others that are equally unsatisfied — so Stage 3 does not treat it as authoritative.

*Phase 2 (best-effort optimized selection).* A separate `Optimize` instance re-asserts the budget constraint and searches for the tactic selection that jointly (a) maximizes how many required quality attributes are covered and (b) minimizes the caller-weighted cost of the trade-offs incurred by the selected tactics, always returning a concrete selection even when Phase 1 found the instance strictly infeasible. These two objectives are declared under Z3's `priority="lex"` lexicographic mode as separate `add_soft` groups, rather than combined into one weighted-sum objective — a distinction we verified matters empirically. An earlier, weighted-sum formulation used a single large constant to bias coverage above trade-off cost, but because trade-off weights are caller-supplied with no upper bound, a tactic selection accumulating enough trade-off weight could mathematically outvote a required attribute's coverage regardless of the bias constant's size (a concrete repro during development: four trade-offs at weight 300 each outweighed a coverage bias of 1000). Lexicographic priority makes quality-attribute coverage strictly dominate trade-off avoidance independent of weight magnitude, which we consider the only encoding that is safe for caller-supplied weights whose scale cannot be bounded in advance. The resulting `covered_quality_attributes` / `uncovered_quality_attributes` split from this phase, not Phase 1's unsat core, is therefore the sound and complete signal of what still needs fixing.

If Phase 1 finds the instance infeasible, Stage 3 constructs a repair prompt that names, for every uncovered quality attribute, the concrete catalog tactics that would cover it, and asks the LLM to revise the candidate within the same tactic budget. This is re-checked against the solver, and the loop iterates up to `max_repair_iterations` times, tracking the best (highest-coverage) attempt seen across iterations. If the budget is never satisfiable within the iteration cap, Stage 3 returns the best attempt found together with an explicit caveat rather than either silently accepting an infeasible decision or discarding an otherwise-reasonable candidate; a transient LLM or parse failure during any individual repair attempt is treated as a reason to try the next iteration, not as a reason to abort the loop.

### E. Stage 4: Self-Critique Finalization

Stage 4 combines two independently-computed signals into each quality attribute's final utility score. The *structural* component is deterministic: for a given attribute $q$, it is $1.0$ if $q$ is among the solver-verified decision's covered attributes, minus a fixed penalty of $0.2$ for every incoming trade-off edge from a selected tactic onto $q$ in $G$, floored at $0.0$. This component depends only on Stage 3's already-verified output and involves no further LLM call. The *qualitative* component is a single LLM pass that scores every required attribute on a 0–10 scale and flags any residual weakness in the finalized decision, given the full decision, rationale, and attribute list in one call; parsing this response uses a strict-pattern-first, tolerant-pattern-fallback matcher (Section III-F) so that a loose regex cannot lock onto a prose preamble instead of the real scored line further down the response, and the call is retried up to three times on a parse failure for the same reason Stage 2's synthesis call is. The two components are combined as $\mathrm{combined}_q = 0.5 \cdot \mathrm{structural}_q \cdot 10 + 0.5 \cdot \mathrm{qualitative}_q$, and the overall decision utility is the mean combined score across all required attributes. The finalized output — the `FinalADR` — carries this scoring together with full provenance: which precedents were retrieved, the complete per-agent deliberation transcript, which tactics were selected and by which repair iteration, and any solver caveat, so the decision is auditable end-to-end rather than presented as an opaque final answer.

### F. Implementation Robustness: Parsing Real, Stochastic LLM Output

A structured-output prompt's hand-written happy-path parser reliably breaks against a real, deployed small language model, and our development process surfaced this repeatedly and concretely rather than as a theoretical concern. Running the full pipeline end-to-end against Qwen2.5-1.5B-Instruct, the reproducible local backbone used throughout this work's experiments, produced five distinct real formatting deviations across Stages 2–4: an extra descriptor word inserted into an otherwise-correct label ("Candidate Decision:" rather than "Candidate:"), a non-literal way of saying no weakness was found ("None identified" rather than the literal string "none"), a fraction-style score ("8/10") that turned out to be this particular model's *default* way of answering a stated 0–10 scale rather than an edge case, a prose preamble a loosely-tolerant regex could lock onto in place of the real scored line appearing later in the same response, and a genuine word substitution ("Candidacy:" for "Candidate:") that no plausible amount of regex tolerance would have anticipated. The first four are addressable by a sufficiently tolerant parser; the fifth is not, and its discovery is what motivated this work's general strategy of pairing every structured-output prompt with a bounded retry (two to three attempts) of the identical prompt on a parse failure, rather than continuing to special-case wording variants indefinitely. Because generation is stochastic, a differently-sampled retry of the same prompt frequently self-corrects a formatting slip the parser did not anticipate; Stages 2, 3, and 4 all apply this pattern. We report this not as a footnote but as a methodological finding in its own right: any system that asks an LLM for structured intermediate output that a downstream deterministic component must consume — as any solver-verified pipeline necessarily does — should validate its parser against real, repeated model output before trusting it in an unattended run, and should treat bounded retry, not parser tolerance alone, as the general-purpose fix once wording deviations stop being enumerable in advance.

## IV. Evaluation

### A. Corpus and Experimental Setup

We evaluate against the "Context Matters" replication package [3] (Zenodo, DOI: 10.5281/zenodo.18370195), derived from the same mining study that provides Stage 1's retrieval corpus [6]. After parsing, the corpus yields 6,173 individual ADR files across 883 repository folders (median 4, mean 6.76, maximum 129 files per repository, confirming the corpus's documented per-repository sparsity). Each folder carries an extraction-confidence status from the package's own `dataset_index.json`; we restrict our held-out evaluation set to the 750 folders marked `Verified`, so that ground-truth quality is not itself a confound. Following this corpus's own upstream precedent — its companion sub-study is explicitly titled "ADR Generation from Titles" — we construct each held-out query as the record's title (standing in for a stated decision context) and treat the record's full body as ground truth, matching the task formulation the retrieval-only baseline below is itself drawn from.

The retrieval index embeds all 6,173 records with a sentence-embedding model (`all-MiniLM-L6-v2`) into a cosine-similarity nearest-neighbor index; a held-out query record is always excluded from its own retrieval results. The local, reproducible LLM backbone used for deliberation, repair, and critique throughout is Qwen2.5-1.5B-Instruct, run via direct `AutoModelForCausalLM.generate()` calls under CUDA.

### B. Baselines and Ablation

We compare CADENCE (`cadence_full`) against three systems, each isolating a subset of CADENCE's stages so that every stage's marginal contribution can be attributed:

TABLE I
Pilot Verification Run ($N = 3$, `TACTIC_BUDGET=4`, All 5 Quality Attributes Required)

| System | BERTScore F1 | BLEU | ROUGE-1 F1 | METEOR |
|---|---|---|---|---|
| `zero_shot` | 0.810 | 1.05 | 0.220 | 0.244 |
| `retrieval_only` | 0.819 | 1.71 | 0.216 | 0.250 |
| `multiagent_no_solver` | 0.805 | 1.57 | 0.143 | 0.111 |
| `cadence_full` | 0.801 | 0.59 | 0.138 | 0.102 |

`cadence_full`: constraint-satisfaction rate 0.00, average repair iterations 2.00 (see discussion below).

TABLE II
Scaled Evaluation Run ($N = 15$, Two `TACTIC_BUDGET` Conditions) — *Placeholder, Pending Background Run*

| System | BERTScore F1 | BLEU | ROUGE-1 F1 | METEOR |
|---|---|---|---|---|
| *[to be filled in from evaluation_results_scaled.json]* | | | | |

- **Zero-shot** (`zero_shot`): a single LLM call given only the decision context, with no retrieval, deliberation, or verification — the Dhar-et-al.-style baseline [1].
- **Retrieval-only** (`retrieval_only`): the decision context plus Stage 1's top-$k$ retrieved precedent titles, with no deliberation or verification — the Context-Matters-style baseline [2].
- **Multi-agent, no solver** (`multiagent_no_solver`): Stages 1–2 in full (retrieval and quality-attribute deliberation), but without Stage 3's solver verification or Stage 4's critique — the ablation that most directly probes what solver verification adds beyond unverified multi-agent debate, and the system most structurally comparable to MAAD's [4] multi-agent premise without its functional-role/LLM-judge design.

Every retrieval-using system routes through the same leakage guard, so no system ever retrieves or is scored against its own query record.

### C. Metrics

We report the four standard generation-quality metrics used in the corpus's own upstream evaluation work, for direct comparability: BERTScore F1, sentence-level BLEU, ROUGE-1 F1, and METEOR, each computed against the held-out record's real, human-authored body text. In addition, and only reportable by a solver-verified system, we report *constraint-satisfaction rate* (the fraction of held-out items for which Stage 3 reached `is_feasible=True`) and *average repair iterations* (how many repair-loop rounds were needed).

### D. Results

Table I reports a small-$N$ pilot run used to verify the full pipeline and evaluation harness end-to-end before committing to a larger run; we retain it here as a sanity-checked, fully real (non-simulated) data point, and report our larger-scale results in Table II.

The pilot run's `cadence_full` constraint-satisfaction rate of 0% is an expected consequence of its deliberately tight parameters (`tactic_budget=4` while requiring coverage of all 5 quality attributes against a tactic catalog in which every tactic supports exactly one attribute), not a defect in Stage 3: this configuration is mathematically infeasible by construction, and Table II's achievable-budget condition ($B = 5$) is designed to demonstrate `cadence_full` reaching `is_feasible=True` in practice, with the tight-budget condition ($B = 2$) retained specifically to demonstrate graceful degradation on purpose rather than by accident.

Across both baselines and CADENCE, BERTScore F1 is nearly flat (0.801–0.819 in the pilot run) while ROUGE-1 and METEOR are noticeably lower for the two deliberation-based systems (`multiagent_no_solver` and `cadence_full`) than for the two single-pass systems. We discuss why in Section V.

## V. Discussion

### A. The BERTScore-versus-Surface-Overlap Asymmetry

Stage 2's synthesis prompt deliberately asks for a terse, one-or-two-sentence decision plus a one-paragraph rationale, so that Stages 3 and 4 can parse it reliably as a structured intermediate representation — not because a terse decision is a better decision, but because a downstream constraint solver cannot reason about open-ended prose. The `zero_shot` and `retrieval_only` baselines, by contrast, are prompted for open-ended "write an ADR" output, which is closer in length and register to the real, often substantially longer, reference ADR bodies that ROUGE-1 and METEOR reward via n-gram and word overlap. This is a length and register mismatch introduced by our own prompt design choices, not a difference in decision quality, and Table I's numbers show it plainly: BERTScore, a semantic-similarity metric computed over contextual embeddings rather than surface token overlap, is comparatively insensitive to this mismatch and remains within 0.02 across all four systems, while ROUGE-1 and METEOR diverge by as much as 0.15 and 0.15 respectively between the shortest and longest-output systems. We consider BERTScore the fairer metric for comparing CADENCE against baselines under this evaluation design, and we flag this explicitly rather than let a reader infer a quality gap from ROUGE/METEOR alone that the underlying decisions do not actually exhibit. We believe this asymmetry is not specific to CADENCE: any structured, multi-stage LLM pipeline whose intermediate representation is deliberately concise — for parseability, for solver-compatibility, or for any reason other than matching a free-text reference's length — will be penalized by surface n-gram metrics relative to a single-pass system prompted for open-ended prose, independent of whether the structured pipeline's underlying decision is better, worse, or equivalent. Evaluations of future solver-verified or otherwise structured-output LLM pipelines should report a semantic-similarity metric alongside, not instead of, surface overlap metrics for exactly this reason.

### B. Threats to Validity

Our held-out evaluation set is drawn from a single corpus (a mining study of open-source ADR adoption); results may not transfer to proprietary or non-open-source architectural practice, where decision drivers, review culture, and documentation discipline can differ substantially. The local LLM backbone (Qwen2.5-1.5B-Instruct) is small by current standards, chosen for reproducibility rather than peak capability; we expect stronger backbones to shift absolute numbers, though we have no reason to expect the qualitative pattern of stage-wise contribution to invert. Our tactic catalog (25 hand-curated tactics across 5 quality attributes) is deliberately small and each tactic supports exactly one attribute in our current encoding, which sets a hard ceiling on achievable coverage under a tight budget by construction; a larger or multiply-supporting tactic catalog would relax this ceiling and merits future work. Finally, our qualitative critique score (Stage 4) is itself LLM-generated. We take seriously the documented finding that LLMs asked to self-correct their own reasoning without external grounding often fail to improve, or even degrade, an already-correct answer [23]; this is precisely why Stage 4's qualitative component is deliberately blended with a deterministic structural component computed from Stage 3's already solver-verified output rather than trusted alone — the qualitative pass critiques a decision that has already been externally checked, not one it is asked to validate unaided. Even so, this blending is a mitigation, not a proof that the qualitative component is reliable, and it is not a substitute for human expert judgment, which we did not collect for this submission.

### C. Engineering Lessons for Solver-Verified LLM Pipelines

Beyond the algorithmic contribution, we believe two implementation-level findings generalize beyond this specific system. First, the lexicographic-versus-weighted-sum distinction in Stage 3's optimizer (Section III-D) is not a minor tuning detail: any multi-objective encoding that lets a caller supply unbounded weights for a secondary objective must either bound those weights or use a priority mechanism that cannot be outvoted by scale, or it risks silently failing to enforce what the caller believed was a hard requirement. Second, the retry-on-parse-failure pattern (Section III-F) generalizes to any pipeline stage where a deterministic downstream component consumes LLM-authored structured output: parser tolerance alone does not converge, because real models produce genuinely novel wording deviations no amount of a priori regex generalization anticipates, while a bounded retry of the identical prompt exploits generation's inherent stochasticity to route around exactly those deviations without requiring them to be enumerated in advance.

## VI. Conclusion

We presented CADENCE, a four-stage algorithm for LLM-assisted architectural decision-making that combines retrieval-augmented case-based reasoning, knowledge-graph-grounded adversarial multi-agent deliberation across ISO/IEC 25010 quality attributes, constraint-solver-verified feasibility with an LLM repair loop targeting the solver's unsatisfiable core, and a separately-scored self-critique finalization stage. Unlike prior work, CADENCE's multi-agent deliberation represents competing quality-attribute concerns adversarially rather than sequential functional roles, and its feasibility check is a formal solver rather than a second LLM opinion — differences we argue are not incidental but load-bearing for any system whose output is meant to inform a decision with real operational consequences. We implemented the complete pipeline against a real corpus of 6,173 mined ADRs, evaluated it against three baselines that isolate each stage's marginal contribution, and reported both standard generation-quality metrics and two metrics — constraint-satisfaction rate and repair-loop convergence — that only a solver-verified system can report at all. We further surfaced and explained a BERTScore-versus-surface-overlap metric asymmetry that we believe applies generally to structured, multi-stage LLM pipelines, and reported two implementation-level findings — lexicographic multi-objective optimization under unbounded caller-supplied weights, and bounded retry as the general fix for unenumerable LLM output-format deviations — that we believe generalize to solver-verified LLM systems beyond this one. Future work includes extending the tactic catalog to permit multi-attribute coverage per tactic, evaluating against stronger LLM backbones and a larger held-out sample, and collecting human expert judgments of decision quality to complement the automatic and solver-verified metrics reported here.

## Acknowledgments

The author thanks the maintainers of the "Context Matters" ADR replication package for making a real, richly annotated ADR corpus publicly available, without which this work's retrieval stage and evaluation would not have been possible.

## References

[1] R. Dhar, K. Vaidhyanathan, and V. Varma, "Can LLMs Generate Architectural Design Decisions? – An Exploratory Empirical Study," in *Proc. IEEE 21st Int. Conf. Software Architecture (ICSA)*, 2024. arXiv:2403.01709.

[2] A. Gupta, R. Dhar, D. Feitosa, and K. Vaidhyanathan, "Context Matters: Evaluating Context Strategies for Automated ADR Generation Using LLMs," accepted at *EASE 2026 Research Track*. arXiv:2604.03826.

[3] Anonymous, "Context Matters – Replication Package," Zenodo, 2026, doi: 10.5281/zenodo.18370195.

[4] R. Li, Y. Zhang, X. Zhou, P. Liang, W. Sun, J. Xuan, Z. Jin, and Y. Liu, "MAAD: Automate Software Architecture Design through Knowledge-Driven Multi-Agent Collaboration," arXiv preprint, 2025. arXiv:2507.21382.

[5] X. Zhou, R. Li, P. Liang, B. Zhang, M. Shahin, Z. Li, and C. Yang, "Using LLMs in Generating Design Rationale for Software Architecture Decisions," accepted for publication in *ACM Trans. Softw. Eng. Methodol. (TOSEM)*, 2025. arXiv:2504.20781.

[6] G. Buchgeher, R. Schöberl, P. Geist, J. Dorninger, C. Haindl, and R. Weinreich, "Using Architecture Decision Records in Open Source Projects – An MSR Study on GitHub," *IEEE Access*, vol. 11, pp. 63725–63740, 2023, doi: 10.1109/ACCESS.2023.3287654.

[7] R. Dhar, A. Kakran, A. Karan, K. Vaidhyanathan, and V. Varma, "DRAFT-ing Architectural Design Decisions using LLMs," arXiv preprint, 2025. arXiv:2504.08207.

[8] R. Su, A. Bakhtin, N. Ahmad, M. Esposito, V. Lenarduzzi, and D. Taibi, "Evaluating Large Language Models for Detecting Architectural Decision Violations," arXiv preprint, 2026. arXiv:2602.07609.

[9] ISO/IEC, *ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model*, 2nd ed. Geneva, Switzerland: Int. Organization for Standardization, 2023.

[10] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021, ISBN: 978-0-13-688609-9.

[11] I. Kumara, H. Kaplan, J. Owotogbe, D. A. Tamburri, and W.-J. van den Heuvel, "Large Language Models for Service-Oriented Computing (LLM4SOC): Review and Research Directions," in *Proc. SummerSOC 2025*, Communications in Computer and Information Science, vol. 2602, Springer, Cham, 2026, doi: 10.1007/978-3-032-07313-6_6.

[12] K. Sellami and M. A. Saied, "Contrastive Learning-Enhanced Large Language Models for Monolith-to-Microservice Decomposition," *Empirical Softw. Eng.*, 2025, doi: 10.1007/s10664-025-10732-z. arXiv:2502.04604.

[13] D. Albuquerque, J. Renan, G. Rodríguez, E. Dantas, A. França, M. Perkusich, K. Gorgônio, and A. Perkusich, "From Textual Requirements to Microservice Architectures: A Comprehensive Evaluation of LLM-Based Design Synthesis," arXiv preprint, 2026. arXiv:2607.28307.

[14] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, "Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning," in *Findings of the Assoc. Comput. Linguistics: EMNLP 2023*, pp. 3806–3824, doi: 10.18653/v1/2023.findings-emnlp.248.

[15] T. X. Olausson, A. Gu, B. Lipkin, C. E. Zhang, A. Solar-Lezama, J. B. Tenenbaum, and R. Levy, "LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5153–5176, doi: 10.18653/v1/2023.emnlp-main.313. arXiv:2310.15164.

[16] S. Szeider, "Bridging Language Models and Symbolic Solvers via the Model Context Protocol," in *Proc. 28th Int. Conf. Theory and Applications of Satisfiability Testing (SAT 2025)*, LIPIcs vol. 341, doi: 10.4230/LIPIcs.SAT.2025.30.

[17] Y. Hao, Y. Chen, Y. Zhang, and C. Fan, "Large Language Models Can Solve Real-World Planning Rigorously with Formal Verification Tools," arXiv preprint, 2024. arXiv:2404.11891.

[18] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving Factuality and Reasoning in Language Models through Multiagent Debate," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, pp. 11733–11763, 2024. arXiv:2305.14325.

[19] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, "Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate," in *Proc. 2024 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2305.19118.

[20] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2308.07201.

[21] A. Madaan *et al.*, "Self-Refine: Iterative Refinement with Self-Feedback," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.

[22] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Proc. NeurIPS*, 2023. arXiv:2303.11366.

[23] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large Language Models Cannot Self-Correct Reasoning Yet," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2310.01798.

[24] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020. arXiv:2005.11401.

[25] Y. Tao, Y. Li, Y. Qin, and Y. Liu, "Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches," arXiv preprint, 2025. arXiv:2510.04905.

[26] N. Wiratunga, R. Abeyratne, L. Jayawardena, K. Martin, S. Massie, I. Nkisi-Orji, R. Weerasinghe, A. Liret, and B. Fleisch, "CBR-RAG: Case-Based Reasoning for Retrieval Augmented Generation in LLMs for Legal Question Answering," in *Proc. Int. Conf. Case-Based Reasoning (ICCBR)*, 2024. arXiv:2404.04302.

[27] K. Hatalis, D. Christou, and V. Kondapalli, "Review of Case-Based Reasoning for LLM Agents: Theoretical Foundations, Architectural Components, and Cognitive Integration," arXiv preprint, 2025. arXiv:2504.06943.